

语言的开发速度及 C/C++ 等编译型语言的性能与安全性。Go 语言在系统编程、网络编程、并发编程等领域有十分广阔的应用前景。

本章主要讲述了 Go 语言编译器的安装;集成开发环境 LiteIDE 的设置与使用;Go 语言的 Hello World 程序;在命令提示符下编译运行和管理代码的各种工具;Go 语言的两种注释形式。初学者学习 Go 语言时,可尝试阅读其官网提供的各种优质资源,如 Effective Go(https://golang.google.cn/doc/effective_go)。

练习题

1. Go 语言诞生于_____年,其创始人有_____个,其中_____也是 C 语言的创始人之一。
2. Go 语言是编译型语言,还是解释型语言?
3. Go 语言的用途很广泛,如并发编程,请再给出一个例子。
4. 说出 Go 语言的一个主要特性。
5. Go 程序的主函数是什么?
6. 在 Windows 命令提示符下,使用什么命令可以查看 GOROOT 的值?
7. 编写 Go 程序,输出文字“您好,中国!”。
8. 要创建并运行 Go 程序,必须执行的 4 个步骤是什么?
9. Go 语言的注释有几种? 分别是什么? 分别使用什么符号?
10. 在 Windows 命令提示符下,编译并运行 Go 程序的 3 个命令是什么?
11. fmt 是哪个英文单词的缩写? 其包含哪些类型的函数?
12. 查阅资料,写出路径 GOPATH 与 GOROOT 的功能。
13. 下列程序是否有错误? 如有,应该怎样修改? 为什么会发生错误?

```
func main()
{
    fmt.Println("您好,中国!")
}
```

14. 与 C 语言类似,Go 语言也使用分号(Semicolon)作为语句的结束标志,但与 C 语言有所不同,Go 语言依据一条简单的规则判断是否需要插入分号。查阅资料,写出 Go 语言的分号自动插入规则。
15. 查阅资料,说出 Go 编译器自动生成的文件 go.mod 的功能(Go 语言的版本号 ≥ 1.11)。
16. 查阅资料,回答 Go 语言的主包与 main()函数有何关系。

2.6 小结

本章讲述了 Go 语言的两类数据类型：基本型和组合型。基本型包括布尔型、字节型、数值型、字符串型和 rune 型。数值型又进一步细分为整型、浮点型和复数。Go 语言使用关键字 `var` 声明一个变量。声明变量有 3 种方式，分别是一般形式、批量形式和简短形式。根据变量在程序中所处的位置和作用范围，变量可分为局部变量和全局变量。

为了方便处理中文、拉丁文等非英文字符而提出 `rune` 类型，它等价于整型 `int32`。字符串有双引号和反引号两种形式。单个字符需要用单引号括起来。转义字符是一种特殊形式的字符常量，如换行符 `\n`。本章最后讲述了 `iota` 常量计数器、基本的输入/输出函数，如输出函数 `Printf()`、输入函数 `Scanf()`。

练习题

1. Go 语言的数据类型分为_____和_____。
2. Go 语言的基本数据类型包括布尔型、字节型、_____、_____和_____ 5 种。
3. Go 语言的数值数据类型包括_____、_____和_____ 3 种。
4. 整型又分为_____和_____两种。
5. 有符号整型包括 `int`、`int8`、`int16`、`int32` 和 `int64`，其实，在多数情况下，使用_____就足够了。
6. 用于存放指针的无符号整型是_____。
7. 声明变量使用的关键字是_____。
8. 使用 3 种方式声明两个整型变量 `a` 和 `b`，并分别赋值 1 和 2。
9. 变量声明的简短形式有 3 个限制条件，它们分别是什么？
10. 在 Go 语言中为变量命名必须遵循 3 个条件，它们分别是什么？
11. Go 语言一共有多少个关键字？请写出其中的 3 个。
12. 在 Go 语言中，变量名的书写惯例是采用_____或_____形式。
13. 变量一旦被声明，Go 编译器就会自动为其赋零值。整型、浮点型和布尔型分别对应的零值是什么？
14. 进行多重赋值时，如果想忽略某个值，可以使用什么变量？
15. 根据变量在程序中所处的位置和作用范围，可将变量分为_____和_____两种。
16. 阅读下列代码，写出输出结果。

```
var a int = 10
func main() {
    var a int = 0
    fmt.Printf("a = %d\n", a)
}
```

17. Go 语言的字符分为两种类型，它们分别是什么？

18. 一个整数,只要其值在什么范围,就可以用字符的形式输出?

19. 写出下列代码的输出结果。

```
func main() {  
    s := "我们都是中国人 hurray"  
    n := len([]rune(s))  
    fmt.Println(n)  
}
```

20. 十进制数 20 对应的二进制数、八进制数和十六进制数分别是多少?

21. 声明常量使用的关键字是_____。

22. 浮点数包括_____和_____两种。很小或很大的浮点数最好用_____表示。

23. 布尔常量只有_____和_____两个,分别代表真和假。

24. 字符串常量的两种表现形式是_____和_____。

25. 写出 3 个常用的转义字符。

26. 使用基本的输入/输出函数前,需要先导入的包是什么?

27. 十进制整数、浮点数、字符串和字符在输出时,使用的格式字符分别是什么?

28. 想查看一个数据的类型,需要使用的格式字符是_____。

29. 3 个输出函数 Print()、Printf()和 Println()的区别是什么?

30. 执行下列代码时,分别为参数 name 和 age 赋值"Hui"和 35:

```
func main() {  
    var name string  
    var age int  
    count, err := fmt.Scanf("%s%d", &name, &age)  
    fmt.Printf("name = %s, age = %d\n", name, age)  
    fmt.Printf("输入了%d个参数\n", count)  
    fmt.Print("错误提示信息:", err)  
}
```

(1) 应该怎样输入这两个参数的值?

(2) 代码的输出结果是什么?

31. 写出下列代码的输出结果_____。

```
const(  
    first = iota + 1  
    -  
    third  
)  
func main() {  
    fmt.Println(first, third)  
}
```

如果两个运算符的优先级相同,则按照从左往右的顺序进行计算。如果记不清两个运算符的优先级大小,则可以使用小括号()。

举例:

```
func main() {  
    var a bool  
    a = 2 == 1+1           //等价于 a = (2 == (1+1))  
    fmt.Println(a)        //输出 true  
}
```

有时使用小括号不是为了改变运算顺序,而是为了提高代码的可读性,这是一种很好的编程实践,减轻了程序员记忆运算符优先级的负担。

```
(a < 10) && (b > 20)      //推荐的做法,尽管小括号是多余的  
a < 10 && b > 20          //不推荐
```

3.7 小结

本章讲述了各种运算符的用法,包括算术运算符、关系运算符、逻辑运算符、位运算符等。将赋值符“=”与其他运算符相结合就构成了复合赋值运算符,如“+=”。求解一个表达式的值时,必须按照运算符的优先级从高到低的顺序进行计算(括号除外)。另外,逻辑运算符还有一个短路求值特性,利用这一特性可以编写出更简洁、更高效的代码。

练习题

1. 编程计算下列各表达式的值:

- (1) $5 + 6$
- (2) $5 - 6$
- (3) $5 * 6$
- (4) $10 / 4$
- (5) $10 \% 4$

2. 写出表达式 $a + b - 5$ 的运算符和操作数。

3. 执行下列代码,其输出结果是_____。

```
fmt.Println(100 - 25 * 3 % 4)
```

4. 已知 $x = 3$, 执行 $x * = 2 + 1$ 语句后 x 的值等于_____。

5. 如果数据在计算机内存中用 1 字节存储,则 -5 的补码为_____。

6. 代码 $5 > 3 + 2$ 的执行结果是_____。

7. 代码 `fmt.Println("a" < "b")` 的执行结果是_____。

8. 代码 `fmt.Println(124 + 3.0)` 的执行结果是_____。

9. 请举例说明短路求值特性。

10. 已知 $x = \text{true}$, $y = \text{false}$, 求下列各表达式的值:

(1) $x \mid\mid y$ (2) $x \&\& y$ (3) $!x$ 11. 已知 $a = 6, b = 5$, 求下列各表达式的值:(1) $a \& b$ (2) $a \mid b$ (3) $a \wedge b$ (4) $\wedge a$ (5) $a >> 1$ (6) $b << 1$

12. 写出下列代码的输出结果:

```
func main() {  
    a, b := 5, 2  
    c := (a != 0 && (b/a) > 0)  
    fmt.Println(c)  
}
```

13. 编写程序, 输出一个三位数 254 的个位数字、十位数字和百位数字。

14. 阅读下列代码, 写出程序的输出结果:

```
func main() {  
    var x, y uint8 = 5, 3  
    fmt.Println(x & y)  
    fmt.Println(x | y)  
    fmt.Println(^x)  
    fmt.Println(x ^ y)  
    fmt.Println(x &^ y)  
    fmt.Println(x << 1)  
    fmt.Println(x >> 1)  
}
```

举例:

```
func main() {  
    for i, val := range "abc" {  
        fmt.Println(i, string(val))  
    }  
}
```

上述代码的输出结果:

```
0 a  
1 b  
2 c
```

4.6 小结

本章讲述了 Go 语言的控制结构,包括选择结构和循环结构。选择结构包括单分支、双分支和多分支 3 种形式。switch 开关语句也是一种选择结构,而且是多分支选择结构。与大多数编程语言不同,Go 语言只支持一种 for 循环,而不支持 while 和 do-while 循环。goto 语句是无条件跳转语句,它通常与语句标号结合使用,可实现从多层循环内部快速跳出来的功能。for-range 循环是一种键值循环。

break 语句能提前终止包含它的循环体的执行过程。break 语句一般用在 for 循环、switch 语句和 select 语句中。continue 语句只是跳过循环体中它后面的剩余语句,并开始下一轮循环过程。break 语句与 continue 语句都能与语句标号结合使用。

练习题

1. 选择结构有 3 种使用形式,分别是单分支结构、双分支结构和_____。
2. 当 else if 分支的数量过多时,通常采用_____语句。
3. 默认情况下,每个 case 语句的末尾都隐含有一个_____语句。
4. 在 Go 语言中,要想模拟 C 语言的 switch 语句的执行流程,需要在每个 case 语句的末尾添加一个关键字_____。
5. Go 语言中的 switch 语句,其每个 case 后面可以有多个表达式,这些表达式之间用_____间隔。
6. 当 switch 后面没有表达式时,编译器会自动为其添加一个逻辑值_____。
7. 在 Go 语言中,每个 case 后面的表达式可以是_____。
8. for 后面的 3 个表达式,相互之间用_____分隔。
9. for-range 循环是一种_____循环。
10. 已知 x 与 y 之间的对应关系,如下表所示:

x	$x < 0$	$0 \leq x < 5$	$x \geq 5$
y	0	x	$3x - 5$

用户输入 x 的值, 程序计算并输出对应的 y 值(保留 2 位小数)。

11. 使用 for 循环计算 $1+2+\dots+100$ 的值。

12. 使用 for 循环输出 1, 2, 3, 5 四个正整数。

13. 求 $[0, 200]$ 范围内能被 13 整除的最大正整数。

14. 求 $[1, 100]$ 范围内能被 7 整除, 但不能被 5 整除的所有整数。

15. 写出下列代码的输出结果_____。

```
var s int
for i := 1; i <= 10; i += 2 {
    s += i
}
fmt.Print(s)
```

16. 编程输出所有的“水仙花数”。所谓“水仙花数”, 是指一个三位的十进制整数, 其各个数位上数字的立方和恰好等于这个整数本身。例如, 153 是水仙花数, 因为 $153 = 1^3 + 5^3 + 3^3$ 。

17. 鸡兔同笼问题: 现有鸡和兔共 30 只, 脚 90 只, 问鸡和兔各有多少只?

18. 输入两个正整数 m 和 n , 求其最大公约数 gcd(Greatest Common Divisor) 和最小公倍数 lcm(Least Common Multiple)。

19. 如果一个数恰好等于它的因子之和, 则这个数就称为“完数”。例如, 6 的因子有 1、2、3, 而 $6 = 1 + 2 + 3$, 因此 6 是“完数”。编程输出 1000 以内的所有“完数”。

20. 使用嵌套循环, 按照下面的格式打印字符 \$:

```
$
$$
$$$
$$$$
$$$$$
```

21. 阅读下列代码, 写出程序的输出结果_____。

```
func main() {
    Loop:
        for _, ch := range "a b\nc" {           //字符串由 a、空格、b、换行符和 c 组成
            switch ch {
                case ' ':                       //跳过空格
                    break
                case '\n':                       //在换行符处终止
                    break Loop
                default:
                    fmt.Printf("%c", ch)
            }
        }
}
```

参的值来自实参。Go 语言支持函数返回多个值,从函数中返回值使用关键字 `return`。与形参名一样,Go 语言支持对返回值进行命名。`lambda` 函数又称为匿名函数,它没有函数名,只有函数体。定义在一个函数内部的匿名函数又叫作闭包,它是连接匿名函数内部与外部的桥梁。

`defer` 语句是延迟处理语句。在 `defer` 语句所属的函数返回之前,延迟处理语句将按照后进先出的顺序执行。`defer` 语句一般用于释放某些已分配的系统资源,如关闭文件。递归函数在其函数体内部调用它自身,它由终止条件和递归条件两部分组成。Go 语言也支持可变长度参数。也就是说,一个函数可以接收任意数量的实参。方法的接受者可为任意类型,如 `int`、结构体。

练习题

1. 写出函数的定义。
2. 在 Go 语言中定义函数时使用的关键字是_____。
3. Go 程序的入口函数是_____。
4. 一个 Go 程序可以包含_____个 `init()` 函数。
5. 写出 `f()` 的函数签名_____。

```
func f(x float64) (y int, z float64) {  
    return 1, 2  
}
```

6. 写出下列函数的返回值_____。

```
func namedRetValues() (x, y int) {  
    x = 2  
    y = 1  
    return y, x  
}
```

7. 定义只有一个形参 `x` 的匿名函数,计算并返回形参 `x` 的平方,使用实参 10 调用该函数,输出该匿名函数的返回值。
8. 通常情况下,延迟处理语句的应用场景是什么?
9. 写出下列代码的执行结果_____。

```
func f(x int) {  
    fmt.Println(x)  
}  
  
func main() {  
    defer f(5)  
    fmt.Println("in main")  
}
```

10. 写出递归函数的定义。
11. 递归函数由_____和_____两部分组成。
12. 定义一个递归函数 `fact()`,计算 `n!`,规定 `0! = 1`。

13. 在 LiteIDE 中,怎样查看一个函数或方法的定义?

14. 定义函数 `prime()`,判断一个整数是否为素数,若是,则返回 `true`,否则返回 `false`。

15. 定义函数 `f()`,实现一个分段函数的计算,如下表所示。

x	$x < 0$	$0 \leq x < 5$	$5 \leq x < 10$	$x \geq 10$
y	0	x	$3x-5$	$0.5x-2$

16. 不使用 `...int`,编程实现“可变长度参数”一节中的 `multiply()`函数。

17. 查阅资料,总结使用关键字 `import` 导入包的 3 种方式。

18. 如果只想调用一个包包含的 `init()`函数,如 `image/PNG`,则使用什么方式导入包?

19. 阅读程序,写出下列代码的输出结果。提示:在函数 `f()`中,延迟函数 `untracing()`的参数 `tracing("f")`先执行,然后再执行语句 `fmt.Println("in f()")`;在函数 `g()`中同理。

```
func tracing(s string) string {
    fmt.Println("Entering:", s)
    return s
}
func untracing(s string) {
    fmt.Println("Leaving:", s)
}
func f() {
    defer untracing(tracing("f"))
    fmt.Println("in f()")
}
func g() {
    defer untracing(tracing("g"))
    fmt.Println("in g()")
    f()
}
func main() {
    g()
}
```

20. 阅读程序,写出下列代码的输出结果。

```
package main
import "fmt"
func visit(lt []int, f func(int)) {
    for _, val := range lt {
        f(val)
    }
}
func main() {
    //使用匿名函数打印切片中的元素
    visit([]int{2, 1, 4, 3}, func(val int) {
        fmt.Println(val)
    })
}
```

21. 使用闭包创建一个偶数生成器 `evenGenerator()`。

```

        fmt.Println(item.Value)
    }
}

```

上述代码的输出结果：

```

1
2
3
5

```

3. 查找与修改元素

查找元素需要使用 for-range 循环遍历列表,而修改元素可通过插入新元素与删除旧元素相结合的方式实现。列表支持的其他方法如表 6-3 所示。

表 6-3 列表支持的其他方法

方法名	功 能	方法名	功 能
lt.Front()	列表 lt 的第一个元素	lt.Back()	列表 lt 的最后一个元素
item.Next()	元素 item 的下一个元素	item.Prev()	元素 item 的上一个元素(previous)
lt.Init()	清空列表 lt	lt.Len()	列表 lt 的长度

Go 语言的其他数据容器还包括 container/heap 堆(实现了优先级队列)和 container/ring 循环双向链表等,限于篇幅,本书不再讲述。

6.5 小结

Go 语言提供了 3 种内置的数据容器,如数组、切片和投影。数组由相同类型的值构成,其长度是不可变的。切片是一种长度可变的数组。创建切片有 3 种方法:使用已有对象创建;通过声明一个未指定大小的数组创建;使用 make() 函数创建。切片有长度和容量两个指标。切片支持的操作包括追加元素 append()、复制切片 copy() 和删除元素。

投影是一种无序的键值对集合。投影以键作为索引,而数组和切片以下标作为索引。投影声明以后必须使用 make() 函数进行初始化才能使用。遍历投影需要使用 for-range 循环。与数组和切片类似,列表也由一系列值构成。不过,列表的元素类型可以不相同。声明列表的方法有两种:一种是使用函数 New();另一种是使用关键字 var。

练习题

1. Go 语言内置的数据容器有几种? 分别是什么?
2. Go 语言内置的数据容器有几种是序列型的? 分别是什么?
3. 声明长度为 10 的整型数组 array,并将它的前 3 个元素赋值为 2、1、3。
4. 写出下列代码的输出结果_____。

```
a := [...]int{2, 1, 3}
```

```
fmt.Println(len(a))
```

5. 两个数组相等必须同时满足几个条件？它们分别是什么？

6. 写出下列代码的输出结果_____。

```
func main() {  
    var letters [3]string  
    letters[0] = "c"  
    letters[1] = "b"  
    letters[2] = "a"  
    for k, v := range letters {  
        fmt.Print(k, v)  
    }  
}
```

7. 声明一个 4 行 2 列的字符串数组 array。

8. 写出下列代码的输出结果_____。

```
func main() {  
    var a [5]int = [5]int{3, 1, 2}  
    fmt.Println(a)  
}
```

9. 写出下列代码的输出结果_____。

```
func main() {  
    array := [3][2]int{1: {0: 3}, 2: {1: 2}}  
    fmt.Println(len(array))  
}
```

10. 切片与数组最主要的一个不同点是_____。

11. 创建切片有几种方法？分别是什么？

12. 分别使用什么函数，能得到切片的长度和容量？

13. 编写代码实现：为切片 x 追加一个新切片[]int{3, 1, 2}。提示：需要将新切片解包。

14. 给出投影的定义。

15. 声明一个投影：var nameScoreMap map[string] float64, 该投影中值的类型是_____, 此时 nameScoreMap = _____。

16. 写出下列代码的输出结果_____。

```
func main() {  
    var m1 = map[string]int{"b": 2, "a": 1, "c": 4}  
    var total int = 0  
    for _, v := range m1 {  
        total += v  
    }  
    fmt.Println(total)  
}
```

17. 编写代码实现：删除投影 `numsMap` 的键值对 `"two": 10`。
18. 在 Go 语言中既然有数组，为什么还要引入列表呢？
19. 声明列表的方法有几种？分别是什么？
20. 编程实现：在列表 `lt` 的头部增加元素 `5`，在其尾部依次增加 3 个元素 `"b"`、`"c"`、`"a"`，遍历该列表，输出其元素的值。

```
a1 := Animal{"cat"}
a2 := Animal{"cat"}
if a1 == a2 {
    fmt.Println("They are equal.")
} else {
    fmt.Println("They are not equal.")
}
```

上述代码的输出结果:

They are equal.

另外,还可以用结构体指针处理链表等问题,限于篇幅,本章不再讲述。

7.7 小结

Go 语言将那些具有内在联系的、不同类型的数据组合在一起,从而形成了一种特殊的数据结构,这种数据结构叫作结构体。Go 语言中没有类、继承等面向对象的概念。Go 语言使用关键字 `type` 和 `struct` 定义结构体。结构体由若干字段组成,函数也可以作为字段使用。没有名称的结构体叫作匿名结构体,没有名称只有类型的字段叫作匿名字段。

如果一个结构体的字段本身也是一个结构体,那么我们称这种类型的结构体为嵌套结构体。Go 语言的嵌套结构体与接口相互配合,实现的功能比面向对象实现的功能扩展性和灵活性更高。与普通的数据类型一样,结构体也可作为实参传递给函数。Go 语言支持定义指向结构体的指针,使用结构体指针可以处理各种链表问题。

练习题

1. Go 语言使用关键字_____和_____定义结构体。
2. 实例化结构体变量的方法有_____种。
3. 没有名称的结构体叫作_____。
4. 请给出结构体的定义。
5. 已知结构体名 `Point`,其包含的两个字段 `x` 和 `y` 都是 `float32` 类型,请给出这个结构体的声明。
6. 写出下列代码的输出结果_____。

```
type Person struct {
    string
    int
}
func main() {
    p1 := Person{string: "Kate", int: 15}
    fmt.Printf("name = %s, age = %d", p1.string, p1.int)
}
```

7. 写出下列代码的执行结果_____。

```

type Fruit struct {
    name string
}
func main() {
    var apple = Fruit{"Apple"}
    fmt.Println(apple)
}

```

8. 已知一个结构体 Fruit, 试着使用 3 种方法实例化一个结构体变量 banana, 其 name 字段的值为 "Banana"。

```

type Fruit struct {
    name string
}

```

9. 在 Go 语言中, 对结构体进行取地址操作 & 可得到结构体的一个实例, 其效果等价于 new() 函数。请以一个具体的例子解释说明。

10. 定义一个结构体 Date, 它包含 3 个字段 year、month、day, 计算该日是本年的第几天。提示: 注意闰年问题。

输入样例:

2023 2 21

输出样例:

52

11. 编写一个函数 calcDays(), 实现上面的计算。由主函数将年、月、日传递给 calcDays() 函数, 计算完成后将天数传回主函数输出。

12. 创建一个结构体 Student, 其包含的字段有姓名 name、学号 numberID、性别 gender、专业 major, 创建该结构体的一个实例 s1, 输出这名学生的专业。

13. 定义一个矩形结构体 Rectangle, 其包含的字段有高度 height 和宽度 width; 再定义两个函数 calcArea() 和 calcPerimeter(), 分别用于计算矩形的面积和周长。

14. 完善下列代码。代码的功能: 创建一个单链表, 它由 3 个节点组成, 然后遍历这个单链表。

```

package main

type Node struct {
    Data int
    Next *Node
}

func showNode(p *Node) {
    for p != nil {
        fmt.Println(*p)
        p = p.Next
    }
}

func main() {

```

```
var head = new(Node)
head.Data = 1
var node1 = _____
node1.Data = 2

head.Next = node1
var node2 = new(Node)
node2.Data = 3
node1.Next = _____
showNode(head)
}
```

上述代码的输出结果：

```
{1 0xc000042260}
{2 0xc000042270}
{3 <nil>}
```

```
func circleArea(r float64) (float64, error) {  
    if r < 0 {  
        e := areaError{r, "半径小于零,面积计算失败!"}  
        return 0, e  
    }  
    return math.Pi * r * r, nil  
}  
  
func main() {  
    r := -5.0  
    area, err := circleArea(r)  
    if err != nil {  
        fmt.Println(err)  
        return  
    }  
    fmt.Printf("Area of circle: %.2f\n", area)  
}
```

上述代码的执行结果： $r = -5$, 半径小于零, 面积计算失败!

另外, 读者还可以使用 `fmt.Errorf()` 函数, 将错误信息进行格式化输出。

8.5 小结

在 Go 语言中, 接口是一种自定义类型, 其中包含若干个方法的签名。接口是抽象的, 不允许创建接口的实例, 但是可以创建接口类型的变量。一种类型只要实现了某个接口, 就可以将该类型的值赋给这个接口类型的变量。声明接口使用关键字 `type` 与 `interface`。接口有两种类型: 一种是空接口; 另一种是非空接口。没有定义任何方法的接口是空接口。空接口没有任何约束条件, 因此任何类型都已实现了空接口。类型断言 (Type Assertion) 和类型开关 (Type Switch) 能给出接口变量中存储的值及其类型。type switch 是一种特殊形式的开关语句, 它比较的是类型, 而不是具体的值。

Go 语言使用 `sort` 包进行排序。排序整型、浮点型和字符串型切片可以直接使用 `Ints()`、`Float64s()` 和 `Strings()` 方法。排序复杂类型的切片, 则需要实现排序接口 `sort.Interface`, 该接口包含 `Len()`、`Less()` 和 `Swap()` 3 个方法。Go 1.8 版本引入的 `sort.Slice()` 方法可以对任何类型进行排序, 而无须自定义实现排序接口。使用 `sort.Slice()` 方法还可以实现多字段排序。

练习题

1. 接口是一个_____概念(抽象的/具体的)。
2. 接口最常见的用途是_____。
3. 声明接口需要使用关键字_____和_____。
4. 本质上, 接口是什么?
5. 接口的类型有几种? 分别是什么?
6. 对整型切片进行排序时, 可以直接使用_____方法。

7. 对字符串切片进行排序时,可以直接使用_____方法。

8. 在实际的生产实践中,一个接口包含的方法通常不超过几个?

9. 编写代码,声明一个空接口类型的变量 i。

10. 已知 f() 的函数定义如下:

```
func f(a, b int) string {  
    return "in f()"  
}
```

编写代码,输出 f() 的函数签名。

11. Go 语言进行排序时需要使用的内置包是_____。

12. 写出 sort 排序包中 Slice() 的函数签名。

13. 自定义排序时,必须实现 sort.Interface 排序接口的 3 个方法: Len()、Less 和_____。

14. 已知 x 是一个接口类型的变量, type 是一个具体的数据类型,给出类型断言的一般形式。

15. 类型断言的返回值有几个? 分别是什么?

16. 写出下列代码的输出结果:

```
var i interface{} = 5.0  
val, _ := i.(int)  
fmt.Println(val)
```

17. 编写代码,声明接口 Sort, 其包含的 3 个方法分别是 Len() int、Less(i, j int) bool、Swap(i, j int)。

18. 有下列代码:

```
var i interface{} = 10  
value, ok := i.(string)
```

请写出变量 ok 的值。

19. 编程实现: 对下列整型切片 vals 进行降序排列并输出。

```
vals := []int{1, 5, 2, 4, 3}
```

20. 已知字符串切片 s:=[]string{"b", "a", "d", "aa", "c"}, 编写程序将其升序排列并输出。

21. 已知投影

```
items:= map[string]int{  
    "b":1,  
    "c":2,  
    "a":3,  
}
```

编写代码,依据键对 items 进行降序排列,然后按键的先后顺序输出该投影。

22. 简单总结一下接口的使用场景。

```
        //blockN
    }
```

举例：

```
func f1(ch chan string) {
    ch <- "f1"
}
func f2(ch chan string) {
    ch <- "f2"
}
func main() {
    ch1 := make(chan string)
    ch2 := make(chan string)
    go f1(ch1)
    go f2(ch2)
    select {
    case val1 := <-ch1:
        fmt.Println("Got:", val1)
    case val2 := <-ch2:
        fmt.Println("Got:", val2)
    }
}
```

上述代码的一次运行结果：

```
Got: f1
```

9.4 小结

协程与通道为并发程序的开发提供了全面支持。协程是一种能与其他函数并发执行的函数。启动协程使用关键字 `go`。实际上，协程是一种轻量级的线程，如有需要，可在程序中创建数千个协程。通道是一种在协程之间执行通信，从而具有同步功能的数据结构。一个协程可以使用通道给另一个协程发送数据，或者从另一个协程接收数据。声明通道使用关键字 `chan`。通道发送或接收数据使用 `<-` 操作符。通道是可以带有缓冲区的。通常使用 `make()` 函数的第二个参数指定缓冲区的容量。遍历通道能实现循环接收数据。

`select` 语句也是一种控制结构，其用法类似于 `switch` 开关语句。它们两者的不同之处在于，`select` 语句只能用于操作通道，其 `case` 子句要么发送数据，要么接收数据。`select` 语句会实时监听通道的状态，一旦有通道准备就绪，就会执行其对应的代码块。如果有多个通道同时准备就绪，`select` 语句就会随机选择一个通道执行。如果所有通道都没有准备就绪，则执行 `default` 语句块。

练习题

1. 写出协程的定义。

2. 给出启动协程 $f(x)$ 的代码, 其实参值为 10。
3. 写出通道的定义。
4. 声明通道 ch 使其只能传输整型数据。
5. 初始化第 4 题声明的通道 ch , 其缓冲区大小为 100。
6. 声明并初始化空接口类型的通道 ch , 其缓冲器的容量为 10。
7. 使用_____函数可得到通道 ch 的当前长度。
8. 使用_____函数可得到通道 ch 的缓冲区容量。
9. 编写代码, 生成 5 个在 $[0, 100]$ 范围的整数。
10. 写出下列代码的输出结果_____。

```
func main() {
    var ch chan int
    ch = make(chan int, 100)
    ch <- 5
    a := <-ch
    fmt.Printf("%d\n", a)
}
```

11. 通道支持的 3 种操作是什么?
12. 为使得 for-range 循环能正常从通道 ch 中接收数据, 发送完数据后不要忘记使用函数_____关闭该通道。
13. 阅读程序, 写出下列代码的执行结果_____。

```
package main
import "fmt"
func main() {
    ch := make(chan string)
    go sendData(ch)
    fmt.Println(<-ch)
}
func sendData(ch chan string) {
    ch <- "喂, 你好"
    fmt.Println("发送完毕!")
}
```

14. 阅读下列代码, 写出其执行结果_____。

```
import (
    "fmt"
    "time"
)
func main() {
    ch := make(chan string)
    go receiveData(ch)
    fmt.Println("没有数据, 接收操作被阻塞")
    ch <- "接收到数据"
    time.Sleep(time.Second) //睡眠 1s
}
```

```
func receiveData(ch chan string) {  
    fmt.Println(<-ch)  
}
```

15. 写出下列代码的执行结果_____。

```
package main  
import "fmt"  
func f1(ch chan string) {  
    ch <- "f1"  
}  
func f2(ch chan string) {  
    ch <- "f2"  
}  
func main() {  
    ch1 := make(chan string)  
    ch2 := make(chan string)  
    select {  
    case val1 := <-ch1:  
        fmt.Println("Got:", val1)  
    case val2 := <-ch2:  
        fmt.Println("Got:", val2)  
    default:  
        fmt.Println("All not ready!")  
    }  
}
```

情况是通过显式地调用 `panic()` 函数。异常发生后程序将立即终止执行过程,只有延迟处理函数会被执行。注意:`panic()`异常函数与 `recover()`恢复函数必须共存于同一个协程,否则不能从异常中恢复。

练习题

1. 错误发生在_____阶段,而异常发生在_____阶段。
2. 在代码中可以使用_____主动抛出异常。
3. `panic()`函数的语法格式是_____。
4. 通常情况下,引发异常有几种情况? 分别是什么?
5. 至少写出两种及两种以上会引发运行时错误的情况。
6. `recover()`函数的语法格式是_____。
7. 如果在同一个函数中既有 `panic()`函数,又有 `defer` 延迟处理的 `f()`函数,则先执行_____。
8. 编写程序,主动抛出被零除异常,然后捕获该异常,并输出信息“被零除”。
9. 举例说明在已关闭的通道上发送或接收数据会引发异常。
10. 写出下列代码的执行结果_____。

```
import "fmt"
func main() {
    defer func() {
        s := recover()
        fmt.Println(s)
    }()
    panic("in panic")
}
```

11. 下列代码是否存在问题? 如果存在,怎样修改?

```
import "fmt"
func main() {
    panic("in panic")
    s := recover()
    fmt.Println(s)
}
```

12. 写出下列代码的执行结果_____。

```
import "fmt"
func main() {
    a := []int{3, 1, 5}
    f(a, 3)
    fmt.Println("程序正常退出")
}
func f(a []int, index int) {
    defer handlePanic()
    if index > len(a)-1 {
```

```
        panic("数组越界访问")
    }
    fmt.Println(a[index])
}
func handlePanic() {
    r := recover()
    if r != nil {
        fmt.Println(r)
    }
}
```

- (1) 打开文件并返回一个文件对象或句柄;
- (2) 使用该句柄执行读写操作;
- (3) 关闭该句柄。

打开文件时需要使用正确的打开模式,如只读模式 `os.O_RDONLY`。读取文件的常用方法有 `os.ReadFile()`、`io.ReadFull()`;写文件的常用方法有 `fmt.Fprintf()`; `os.OpenFile()` 函数可将数据追加到文件的末尾。`Seek()` 函数用于移动文件指针, `os.Remove()` 函数用于删除文件, `Size()` 函数用于获取文件的大小。

文件夹操作主要讲述了 `os` 模块。`os.Mkdir()` 函数用于创建文件夹, `filepath.Glob()` 函数用于搜索与给定模式相匹配的文件名。实际上,大多数程序员只喜欢使用其中的某个或某些模块。读者现在只需记住模块名,等真正需要用时再查阅相关文档。

练习题

1. 什么是文件? 什么是文件夹?
2. 常见的文件类型有几种? 它们分别是什么?
3. 写出操作文件和文件夹常用的包。
4. Go 语言进行文件操作的 3 个主要步骤是什么?
5. 文件的打开模式 `os.O_RDWR` 代表_____。
6. 使用函数_____可以移动文件指针的位置。
7. 使用_____函数创建文件夹;使用_____函数搜索与给定模式相匹配的文件名。
8. 编写代码,将文件 `hui.txt` 的指针移动到距离文件头 10 个字符(不包括)的位置。
9. 打开文本文件 `fun.txt`,从文件头开始读取 5 个字符。文本文件 `fun.txt` 的内容如下。
`Greatest Chinese Dream.`
10. 打开文本文件 `sample.txt`,将下列内容写入该文件。
`hello world.\n` 文本文件的第二行 `\n` 文本文件的最后一行
11. 打开文本文件 `sample.txt`,逐行读取该文件的内容,搜索并输出长度最长的行。
12. 一个文件的访问权限为 0744,写出文件的拥有者、文件的所属组,以及其他用户的访问权限。
13. 使用 `os.OpenFile()` 函数新建一个文本文件 `fun.txt`,将文字“保持战略定力,发扬斗争精神!”写入该文件。
14. 如果在 Windows 命令提示符输入命令 `go run main.go file1 file2`,则 `len(os.Args)` = _____。
15. 使用 `os.ReadFile()` 函数读取并输出文本文件 `fun.txt` 的内容。
16. 编写程序,创建访问权限为 0755 的文件夹 `tmp`。
17. 编写程序,创建多级目录 `2023/5/6`,设置其访问权限为 0755。
18. 编写程序,获取文本文件 `hui.txt` 的大小。

```
func (a A) f3() {} //方法名的首字母小写
func main() {
    a := A{}
    fmt.Println(reflect.TypeOf(a).NumMethod()) //方法 F1() 和 F2()
}
```

上述代码的输出结果:

2

reflect 在生成数据库查询语句等任务中也是很有用的一种技术,限于篇幅,本书不再举例。

13.6 小结

Go 语言拥有丰富的内置包(46 个),这使得程序开发人员可以很容易地编写出既高效又可靠的代码。本章选取了其中 9 个常用的内置包进行初步介绍,分别是 fmt、net/http、encoding/json、io、os、strconv、math、strings 和 reflect 包。

练习题

1. Go 语言内置包的路径是_____。
2. 写出 Go 语言 5 个常用的内置包。
3. 查看环境变量 GOROOT,可以在 Windows 命令提示符下执行命令_____。
4. 编写 Web 应用程序需要使用_____包。
5. 编码和解码 JSON 数据需要使用_____包。
6. _____函数将 Go 数据结构转换为 JSON 数据。
7. strconv 包的功能是什么? 写出其中 3 个常用函数的函数名。
8. 写出数学包 math 5 个常用函数的函数名。
9. 简述 reflect 包的功能。
10. 为了将错误信息进行格式化输出,需要使用_____函数。
11. 编写程序,使用 os 包读取并显示命令行参数。
12. 编写程序,将字符串"3.14"转换为浮点数。
13. 查阅资料,写出一个方便易用的 Go 图形用户界面(Graphical User Interface,GUI)工具包。
14. 写出下列代码的执行结果_____。

```
import (
    "fmt"
    "log"
    "net/http"
)
func main() {
```



```
resp, err := http.Get("http://www.baidu.com")
if err != nil {
    log.Fatal(err)
}
buf := make([]byte, 15)
resp.Body.Read(buf)
fmt.Println(string(buf))
}
```